

Clinic Appointment Booking System

1. Introduction

Healthcare is one of the most essential needs in any society. With the rise in technology, the need to digitize healthcare services such as appointment bookings has become increasingly important. The Clinic Appointment Booking System is a software solution developed in Java to simplify the process of scheduling patient appointments. It reduces paperwork, eliminates booking conflicts, and enhances the patient experience through a digital interface.

2. Overview

This project aims to automate the clinic appointment system where patients can book, view, or cancel appointments through a user-friendly interface. The receptionist or clinic admin can manage appointments, doctor schedules, and patient records. It ensures efficient appointment handling, maintains a record of past visits, and minimizes the chances of double-booking.

3. Problem Statement

Traditional appointment booking in clinics is time-consuming, prone to human error, and lacks real-time availability. Patients may face long queues or repeated follow-ups for scheduling. Similarly, managing doctor availability manually is inefficient. This project solves these issues by introducing an automated appointment system that allows patients to book appointments based on real-time doctor availability.

4. Objectives

- 1) To develop a digital platform for booking and managing clinic appointments.
- 2) To enable patients to check doctor availability and book appointments accordingly.
- 3) To provide an admin panel for managing users, schedules, and appointments.
- 4) To store appointment and user data in a structured and secure way.
- 5) To generate reports of patient visits and appointment history.

5. Technologies Used

Programming Language: Java (JDK 8+)

GUI Framework: Java Swing

Database: MySQL

Backend Connector: JDBC (Java Database Connectivity)

IDE: Eclipse / NetBeans

6. System Requirements

- Hardware Requirements:

Processor: Intel Core i3 or higher

RAM: Minimum 4 GB

Hard Disk: 500 MB

- Software Requirements:

Operating System: Windows 7/8/10 or Linux

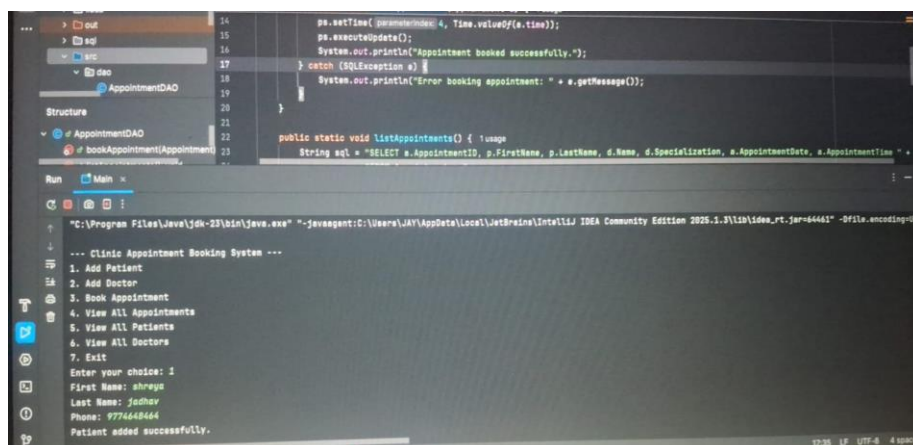
Java Development Kit (JDK)

MySQL Database

Java IDE (Eclipse/NetBeans)

7. Screenshots

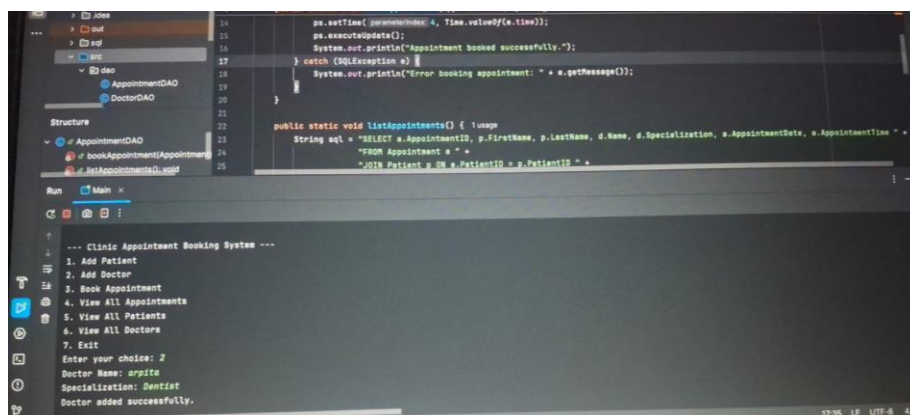
1. Add Patient



```
14 ps.setTime(parameters[4], Time.valueOf(e.time));
15 ps.executeUpdate();
16 System.out.println("Appointment booked successfully.");
17 } catch (SQLException e) {
18     System.out.println("Error booking appointment: " + e.getMessage());
19 }
20
21
22 public static void listAppointments() {
23     String sql = "SELECT a.AppointmentID, p.FirstName, p.LastName, d.Name, d.Specialization, a.AppointmentDate, a.AppointmentTime " +
24         "FROM Appointment a " +
25         "JOIN Patient p ON a.PatientID = p.PatientID " +
26         "JOIN Doctor d ON a.DoctorID = d.DoctorID";
27     ps.setString(1, sql);
28     ps.executeQuery();
29     while(rs.next()) {
30         System.out.println("AppointmentID: " + rs.getInt(1) + ", Patient: " + rs.getString(2) + " " + rs.getString(3) + ", Doctor: " + rs.getString(4) + ", Date: " + rs.getString(5) + ", Time: " + rs.getString(6));
31     }
32 }
```

```
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 1
First Name: ahanya
Last Name: jadhav
Phone: 9774648464
Patient added successfully.
```

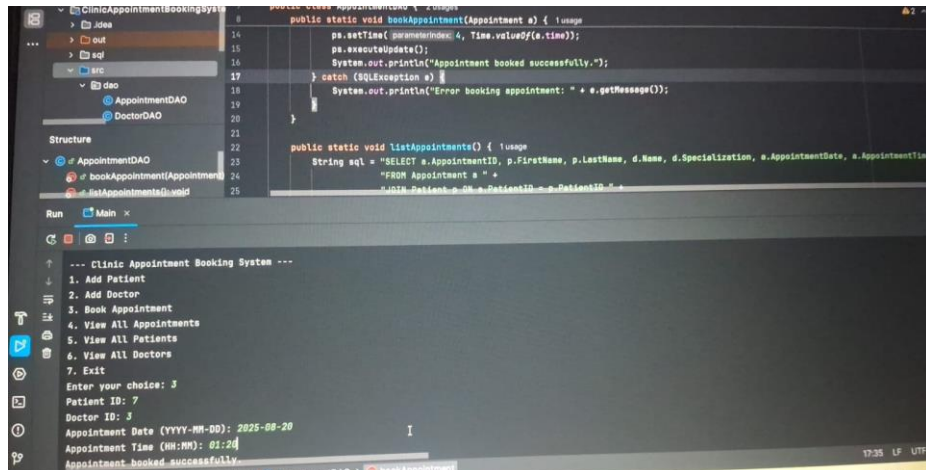
2. Add Doctor



```
14 ps.setTime(parameters[4], Time.valueOf(e.time));
15 ps.executeUpdate();
16 System.out.println("Appointment booked successfully.");
17 } catch (SQLException e) {
18     System.out.println("Error booking appointment: " + e.getMessage());
19 }
20
21
22 public static void listAppointments() {
23     String sql = "SELECT a.AppointmentID, p.FirstName, p.LastName, d.Name, d.Specialization, a.AppointmentDate, a.AppointmentTime " +
24         "FROM Appointment a " +
25         "JOIN Patient p ON a.PatientID = p.PatientID " +
26         "JOIN Doctor d ON a.DoctorID = d.DoctorID";
27     ps.setString(1, sql);
28     ps.executeQuery();
29     while(rs.next()) {
30         System.out.println("AppointmentID: " + rs.getInt(1) + ", Patient: " + rs.getString(2) + " " + rs.getString(3) + ", Doctor: " + rs.getString(4) + ", Date: " + rs.getString(5) + ", Time: " + rs.getString(6));
31     }
32 }
```

```
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 2
Doctor Name: arpitc
Specialization: Dentist
Doctor added successfully.
```

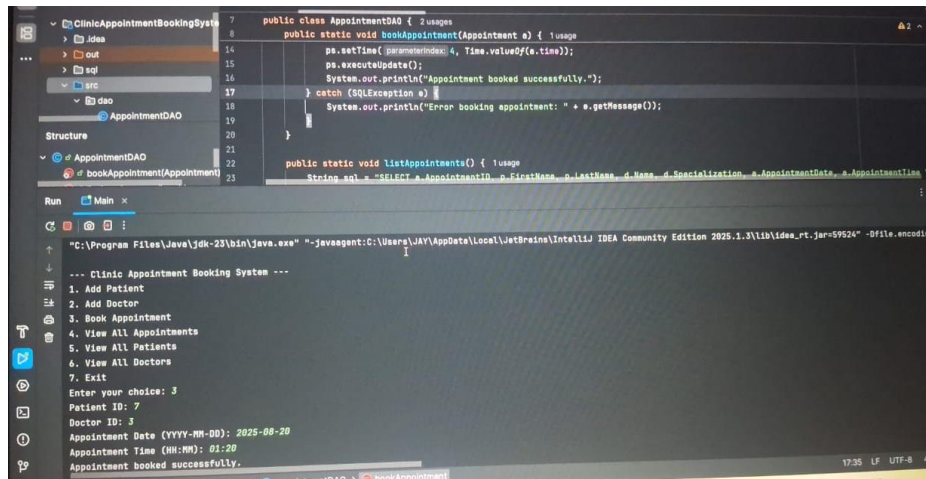
3.Book Appointment



The screenshot shows the IDE with the `AppointmentDAO` class. The `bookAppointment` method is highlighted. The console output shows the following sequence of events:

```
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 3
Patient ID: 7
Doctor ID: 3
Appointment Date (YYYY-MM-DD): 2025-08-20
Appointment Time (HH:MM): 03:24
Appointment booked successfully
```

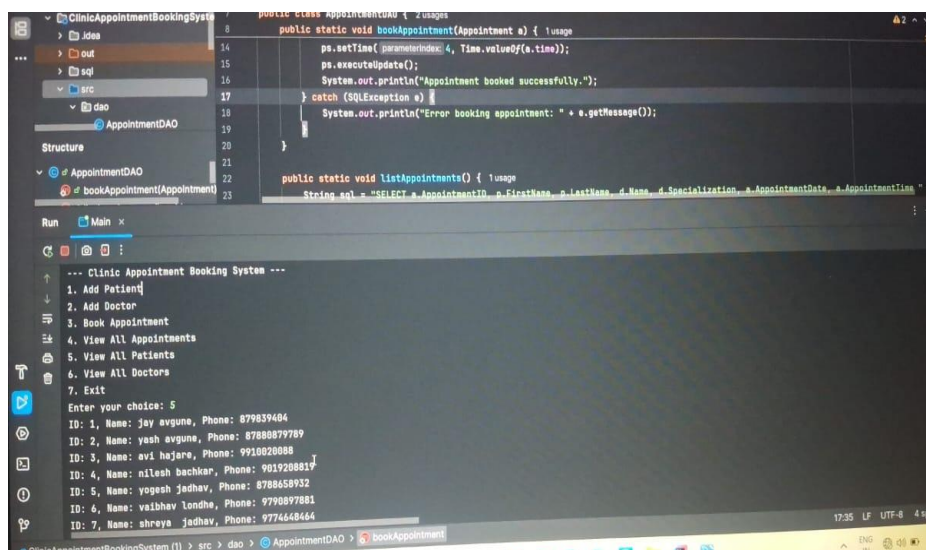
4.View all appointments



The screenshot shows the IDE with the `AppointmentDAO` class. The `listAppointments` method is highlighted. The console output shows the following sequence of events:

```
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 4
Appointment booked successfully
```

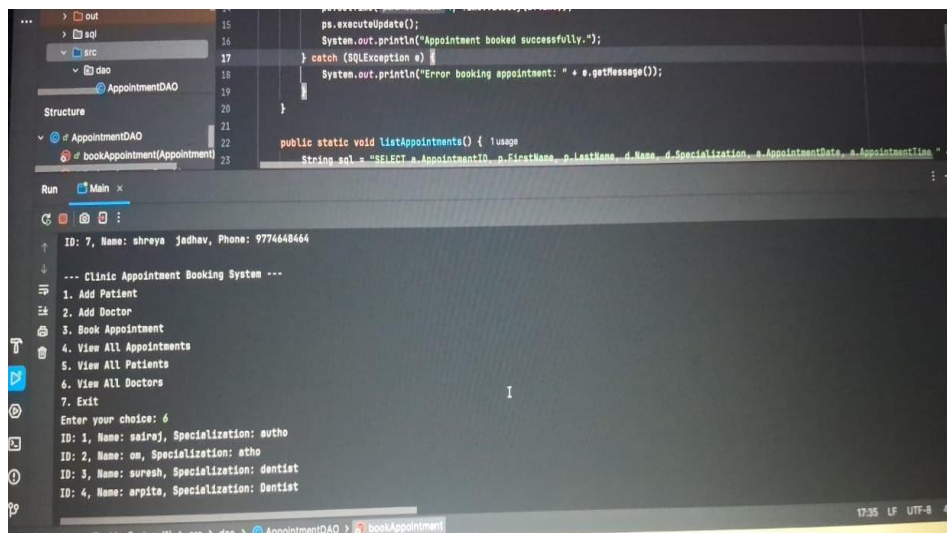
5.View all Patients



The screenshot shows the IDE with the `AppointmentDAO` class. The `listAppointments` method is highlighted. The console output shows the following sequence of events:

```
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 5
ID: 1, Name: Jay avguna, Phone: 879839484
ID: 2, Name: yash avguna, Phone: 87889879789
ID: 3, Name: avi hojars, Phone: 9910020098
ID: 4, Name: nilesh bachkar, Phone: 9019208817
ID: 5, Name: yogesh jadhav, Phone: 8788658932
ID: 6, Name: vaibhav londhe, Phone: 9790897881
ID: 7, Name: shreya jadhav, Phone: 9774648464
```

6.View all Doctors



```
15 ps.executeUpdate();
16 System.out.println("Appointment booked successfully.");
17 } catch (SQLException e) {
18     System.out.println("Error booking appointment: " + e.getMessage());
19 }
20
21 public static void listAppointments() { usage
22     String sql = "SELECT a.AppointmentID, a.FirstName, a.LastName, d.Specialization, a.AppointmentDate, a.AppointmentTime " +
23 }
```

```
Run Main x
ID: 7, Name: shreya jedhav, Phone: 9774648464
--- Clinic Appointment Booking System ---
1. Add Patient
2. Add Doctor
3. Book Appointment
4. View All Appointments
5. View All Patients
6. View All Doctors
7. Exit
Enter your choice: 6
ID: 1, Name: sairaj, Specialization: autho
ID: 2, Name: am, Specialization: atho
ID: 3, Name: suresh, Specialization: dentist
ID: 4, Name: arspite, Specialization: Dentist
```

7.Exit

8. Conclusion

The Clinic Appointment Booking System offers a practical and efficient solution for both patients and clinic staff. By digitizing the process, it saves time, ensures better patient management, and helps clinics operate more smoothly. The project demonstrates the application of core Java concepts, GUI development, and database integration, making it an ideal academic or real-world project.